

به نام خدا

810199421	آریان رجبی قلعه
810100084	محمد امانلو
810100119	سهیل حاجیان منش

گزارش کار آزمایشگاه سیستم های دیجیتال 2 (آزمایشگاه معماری)

گروه یکشنبه 15:30

بهار 1404

1	 به نام خدا
3	 جلسه اول - 19 اسفند 1403
8	 جلسه دوم - 17 فروردین 1404
10	 جلسه سوم - 24 فروردین 1404
11	 جلسه چهارم - 31 فروردین 1404
13	 جلسه پنجم - 7 اردیبهشت 1404
14	 جلسه ششم - 14 اردیبهشت 1404
16	 جلسه هفتم - 28 اردیبهشت 1404
19	 جلسه هشتم - 4 خرداد 1404

نکات مهم: شماره جلسات دقیق نیستند، اما تاریخ آن ها به درستی ذکر شده است

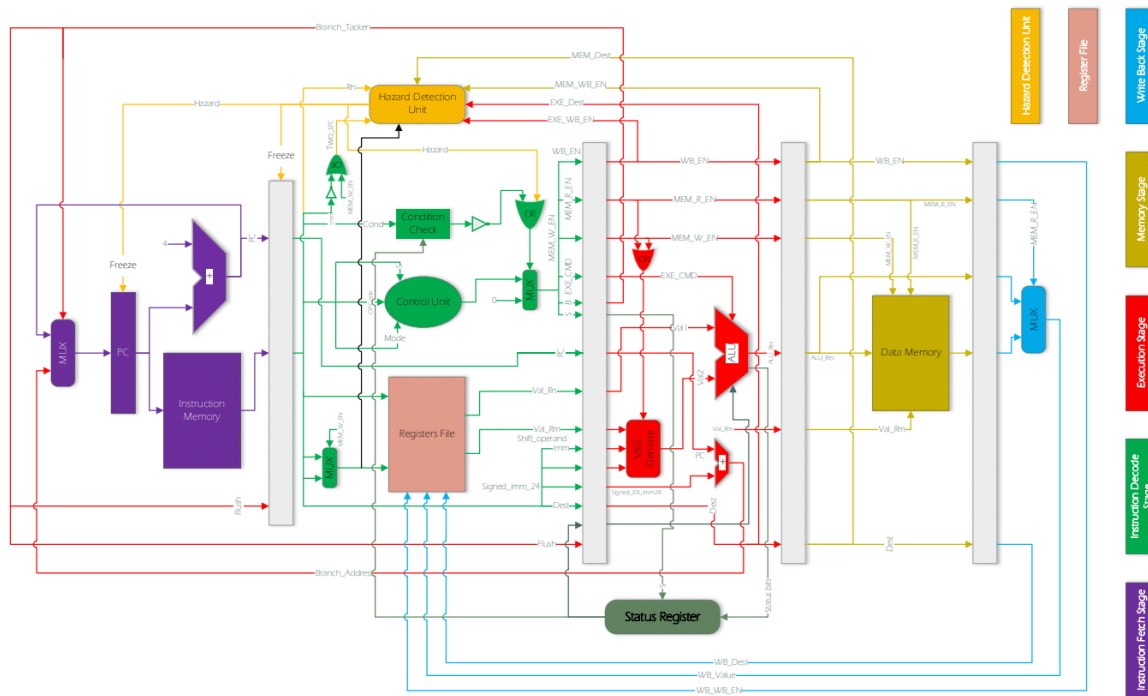
چالش های ذکر شده در گزارش، حاصل یادداشت های در طول آزمایشگاه بوده اند

سورس کد نهایی، همان سورس کد آپلود شده 12 خرداد (فورواردینگ یونیت) می باشد

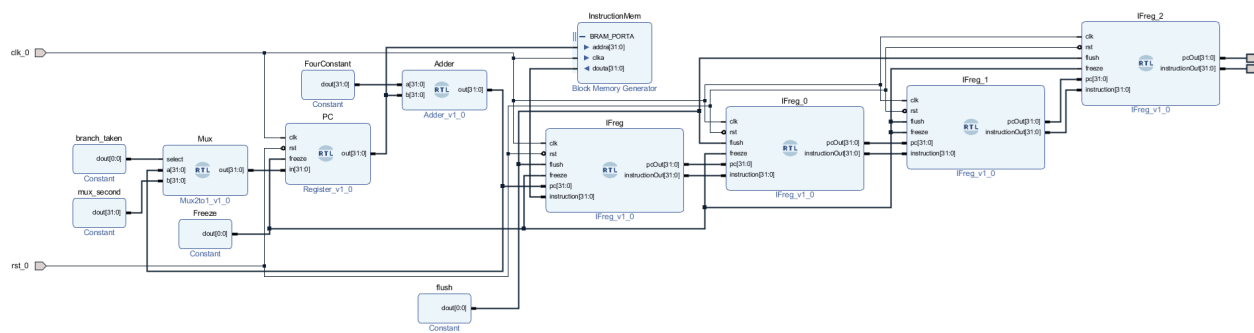
برای ران کردن درست فایل نهایی ویوآدو، فایل مموری را از فیبوناچی (کوییز) به مموری صورت پروژه تغییر دهید، همچنین اگر سورس کدی مشکل رفرنس داشته باشد، در همان آپلود 12 خرداد موجود است

جلسہ اول - 19 اسفند 1403

در جلسه اول، طراحی Instruction fetch stage آغاز شد (بخش بنفش)، در این مرحله از ARM، ماژول PC به عنوان یک counter عمل می کند و دستورات را یکی یکی از حافظه دستورات می خواند، جهت بررسی درستی طراحی، در این فاز هر 4 رجیستر میانی را از IF reg نمونه گرفتیم (instance)، تا یک دستور یک دور کامل در pipeline فرضی بچرخد. (مطابق صورت پروژه)



شکل. طراحی کلی پردازنده ARM



شکل. طراحی اولیه بدون ILA

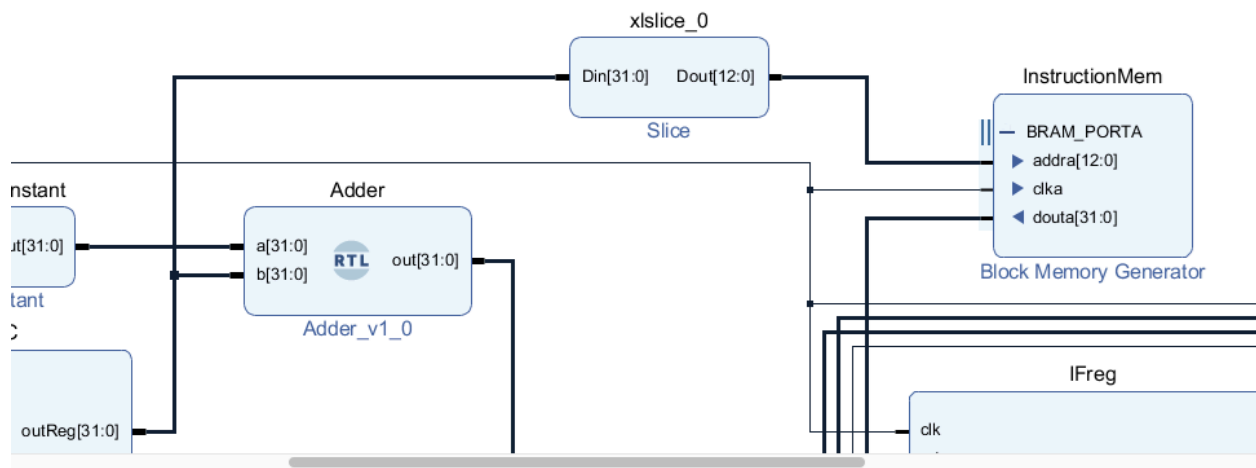
با چالش‌های زیر مواجه شدیم:

1. مشکل در ریست رجیسترها

در ابتدا ریست برخی از ماژول‌های رجیستر به درستی عمل نمی‌کرد. دلیل آن این بود که یک `else` در کد جا افتاده بود و چندین شرط `if` بدون انحصار اجرا می‌شدند. همچنین، در نظر نگرفتن `posedge` برای ریست موجب بروز اختلال شد که پس از تشخیص این مشکل، آن را به کد اضافه کردیم.

2. نیاز به استفاده از Slicer

چون خروجی PC به صورت ۳۲ بیتی بود و ورودی Instruction Memory تنها ۱۳ بیت نیاز داشت، نیاز به افزودن یک slicer بین این دو ماژول بود تا بیت‌های اضافه حذف شوند.



شکل. افزودن slice

3. تأخیر در Instruction Memory

مطابق دستورالعمل طراحی IP مربوط به Instruction Memory، این حافظه دارای دو سیکل تأخیر بود که در طراحی لحاظ شد.

4. استفاده از ILA برای دیباگ

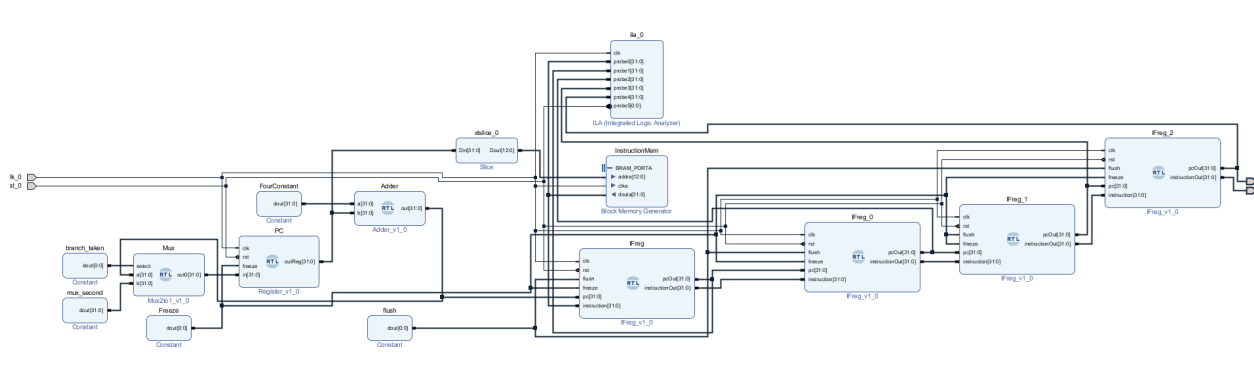
جهت بررسی سیگنال‌ها درون سخت‌افزار، ابزار ILA را به طراحی اضافه کردیم. تریگر ILA به سیگنال ریست متصل شد تا با فشردن کلید ریست، تحلیل از ابتدا شروع شود. همچنین، پنج Probe به خروجی‌های رجیسترها متصل شدند. Probe صفر به خروجی Instruction آخرین رجیستر متصل شد و Probe‌های ۱ تا ۴ به خروجی‌های Stage Register های مختلف.

Component Name `ila_0`

To configure more than 64 probe ports use Vivado Tcl Console

Probe Port	Probe Width [1..4096]	Number of Comparators	Probe Trigger or Data
PROBE0	32	1	DATA
PROBE1	32	1	DATA
PROBE2	32	1	DATA
PROBE3	32	1	DATA
PROBE4	32	1	DATA
PROBE5	1	1	TRIGGER

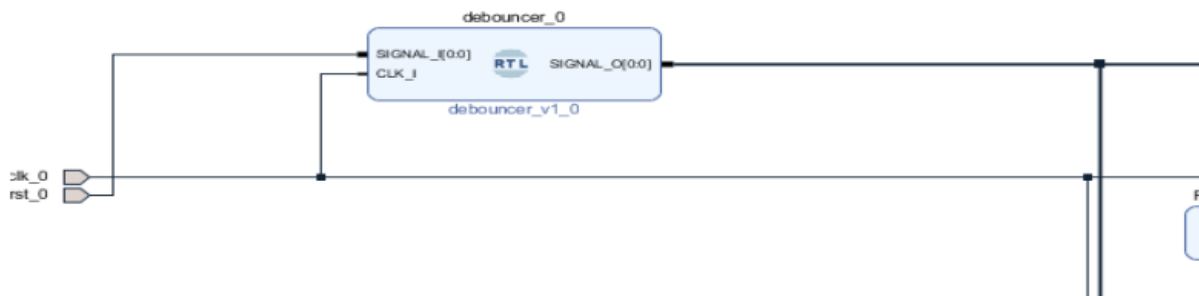
شکل. تنظیمات ILA



شکل. طراحی بعد افزوده شدن ILA

5. دیانسر ریست

تمام سیگنال‌های ریست به خروجی ماژول دیانسر متصل شدند. ورودی دیانسر نیز از یک منبع خارجی تأمین می‌شود. (ورودی دیانسر = ریست فیزیکی روی سخت افزار)



شکل. افزودن debouncer

6. اصلاح در Wrapper

در فایل Wrapper، سیگنال ریست معکوس شد (not شد) تا با posedge کار کند چون سخت افزار negedge بود و اغلب ماژول‌ها با لبه بالارونده ریست تنظیم شده بودند (به جز نوشتن در مموری که در جلسات آینده متوجه می شویم برای رفع دسته ای از Hazard ها می باشد). این فایل توسط Vivado ایجاد می شود.

7. افزودن فایل XDC

فایل XDC به پروژه اضافه شد تا پورت‌های I/O به درستی روی پین‌های FPGA مپ شوند.

8. ساخت Bitstream

پس از اتمام طراحی، فایل بیت‌استریم تولید شد.

9. خروجی اضافی در رجیستر آخر

در رجیستر آخر خروجی‌ای وجود داشت که نباید وجود می‌داشت. این مشکل با حذف آن خروجی برطرف شد.

10. مشکل در نرم‌افزار Vivado

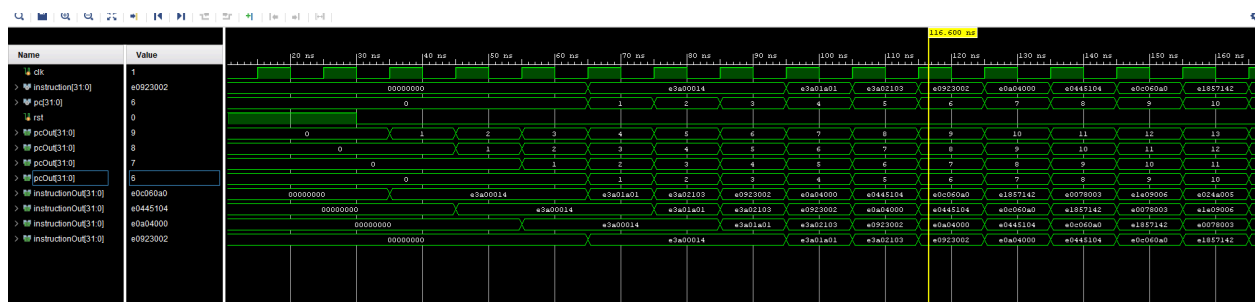
به نظر می‌رسید که طراحی باگ داشت یا فایل پروژه آسیب دیده بود. بنابراین، یک پروژه جدید ساختیم و همه فایل‌ها و تنظیمات را به پروژه جدید منتقل کردیم.

```

22
23 module tb(
24
25 );
26 reg clk;
27 wire [31:0] instruction;
28 wire pc;
29 reg rst;
30 design_1 TopModule
31     (.clk_0(clk),
32      .instructionOut_0(instruction),
33      .pcOut_0(pc),
34      .rst_0(rst));
35
36     always #5 clk = ~clk;
37
38     initial begin
39         clk=0;
40         rst=0;
41         #10 rst = 1;
42         #20 rst = 0;
43         #200 $stop;
44     end
45 endmodule
46

```

شکل. کد تست بنچ اولیه



شکل. نتیجه شبیه سازی - دستورات به ترتیب خوانده می شوند

جلسه دوم - 17 فروردین 1404

در این جلسه، تمرکز اصلی بر روی پیاده سازی فاز Instruction Decode پردازنده بود، این فاز شامل یک رجیستر فایل به عنوان حافظه ای از رجیستر ها برای پردازنده، و control unit پردازنده می باشد، دستور گرفته شده از مرحله IF در این مرحله decode می شود، این decode می تواند شامل حافظه هایی از فایل رجیستر برای خواندن و نوشتن نیز باشد.

چالش ها:

1. نکته نحوی در Verilog

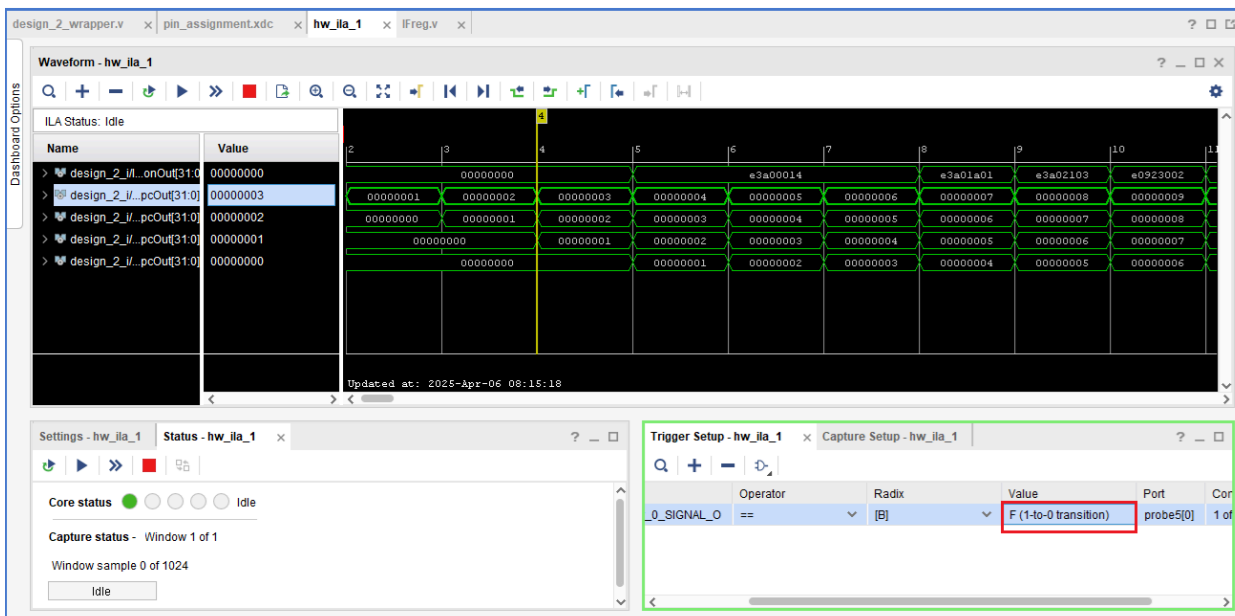
هنگام نوشتن عملیات نات (NOT)، نباید از ! استفاده کرد، بلکه باید از ~ استفاده شود تا عملیات بیتی انجام شود.

2. واژه های رزرو شده

واژه هایی مانند **in**, **select**, **out** در Verilog رزرو شده اند، ولی در برخی موارد بدون تغییر هم به درستی کار کردند.

3. اصلاح Trigger در Program

برای رفع مشکل در زمان program، نوع تریگر را به حالت "From 1 to 0 transition" تغییر دادیم و مشکل برطرف شد.



شکل. اصلاح TRIGGER

4. تحلیل طراحی کنترلر

در طراحی کنترلر ما، نیازی به Slicer برای جدا کردن OPCODE و MODE از Instruction نبود، زیرا داخل خودش تقسیم می شود و مستقیماً instruction را ورودی را می گیرد. در دیزاین کنترلر ما، opcode و mode ... داخل خودش تقسیم می شود و نیازی به slice کردن نیست (مستقیم instruction را ورودی می گیرد)

5. تحلیل طراحی CUMUX (مالتیپلکسر بعد از Control Unit)

در طراحی CUMUX، تمام خروجی‌های Control Unit به آن متصل هستند. اگر سیگنال Selector صفر باشد، تمام خروجی‌ها صفر می‌شوند و در غیر این صورت، مقادیر کنترلر عبور داده می‌شوند.

6. مشکلات مرتبط با Imm

ابتدا خروجی Imm را از CUMUX می‌گرفتیم، ولی متوجه شدیم که بهتر است مستقیماً از Instruction آن را استخراج کنیم. در طراحی نهایی، Imm بدون دخالت CUMUX مستقیماً از Instruction جدا شد.

7. برنامه جلسه بعد

برای جلسه آینده، قرار شد رجیستر مرحله ID (یعنی IDreg) را به Block Design اضافه کنیم.

جلسه سوم - 24 فروردین 1404

در این جلسه بیشتر روی کدنویسی فاز Execute پردازنده تمرکز داشتیم، اصلی ترین ماژول این بخش، ALU می باشد، این ماژول combinational انواع دستورات محاسباتی مثل جمع را انجام می دهد، کد زنی ماژول های combinational به طور کلی چالش های خیلی کمتری دارند، در نتیجه این جلسه با چالش های خیلی کمتری مواجه بودیم.

چالش ها:

1. اشتباه در گرفتن Imm از Multiplexer

در کد ما برای imm از اسلایس استفاده نکردیم چون اشتباهی imm از CUMux میاد (مستقیم وارد شده و خارج شده) و جایگزین اسلایس هست

2. غیرفعال کردن دیبانسر

برای اینکه طراحی روی نرم افزار به درستی کار کند، ماژول دیبانسر را به طور کامل از مدار حذف کردیم. روی سخت افزار باید debouncer رو بزاریم روی ریستارت چون روی سخت افزار، دکمه ریست ممکن است هنگام فشردن چندین پالس ناپایدار (نویز) تولید کند؛ برای همین استفاده از دیبانسر ضروری است تا فقط یک پالس پایدار به مدار اعمال شود

جلسه چهارم - 31 فروردین 1404

در این جلسه، تمرکز اصلی روی اصلاح حافظه‌ها، بررسی خروجی‌ها، پیاده سازی بخش Mem و رفع اشکالات در مراحل EXE و MEM بود:

1. تغییر نوع Instruction Memory به distributed ROM

با توجه به اعلام دستیار آموزشی، نوع حافظه Instruction Memory را از block-based به distributed ROM تغییر دادیم. این کار به دلیل نیاز به عملکرد صحیح در هنگام لود دستورات عمل‌ها انجام شد.

2. علت نیاز به ROM در Instruction Memory

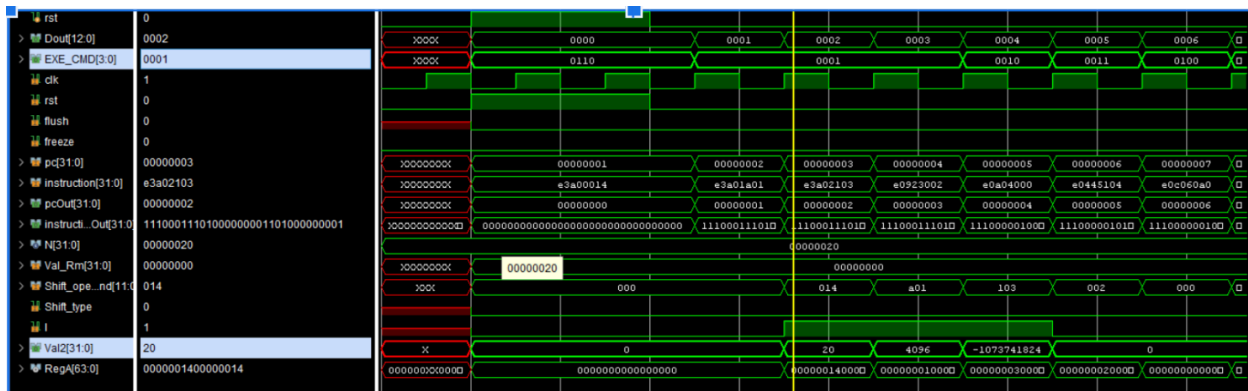
چون خروجی حافظه دستورات عمل از نوع LUT است و دارای یک کلاک تأخیر می‌باشد، اگر از ROM استفاده نکنیم، PC جلوتر از Instruction حرکت می‌کند و هماهنگی بین این دو از بین می‌رود. استفاده از distributed ROM این مسئله را حل کرد.

3. تنظیم نوع حافظه برای Data Memory

برای حافظه داده‌ها، نوع حافظه را RAM انتخاب کردیم تا بتوان عملیات خواندن و نوشتن را انجام داد. در مقابل، حافظه دستورات عمل فقط خواندنی است و باید از نوع ROM باشد.

4. بررسی سیگنال carry-in در ALU

در صورتی که اجرای کد با خطا مواجه می‌شد، یکی از علت‌ها می‌توانست اشتباه بودن مقدار ورودی در carry-in باشد. این ورودی باید از بیت یکم رجیستر وضعیت (status[1]) گرفته می‌شد. اگر ترتیب نوشتن در رجیستر وضعیت نادرست باشد، ALU نتیجه اشتباهی تولید خواهد کرد.



5. تحلیل فایل تست و کدهای EXE و ValGen

در اسکرین‌شات بالا مربوط به تست، بخش EXE_CMD و Val2 هایلایت شده بود. دستور MOV (بر اساس فایل ARM.pdf-00) باید مقدار کامند 0001 را در EXE_COMMAND تولید کند. همچنین خروجی ValGen باید مطابق ستون سمت راست صفحه ۱۷ همان فایل باشد که مربوط به Val2 است.

6. اصلاحات در ماژول ValGen

در مرحله تست ماژول حافظه، خروجی نادرست بود. بعد از بررسی، متوجه شدیم که در ValGen به جای استفاده از [shift_operand[11:0]، باید از [shift_operand[11:7] استفاده می‌شد. همچنین در یکی از caseها در ValGen، بیت‌های مربوط به نوع شیفت اشتباه بودند و باید بیت‌های [6:5] برای تعیین نوع شیفت در نظر گرفته می‌شدند که این نیز اصلاح شد.

7. مشکل در خروجی Mux_cyan و Val2Gen

همچنان طراحی مشکل داشت. طبق بررسی نهایی، خروجی ماژول Mux_cyan در بسیاری از مواقع صفر بود، که علت آن صفر بودن خروجی ALU تشخیص داده شد. در نتیجه، مقدار نادرستی در رجیستر فایل نوشته می‌شد (مثلاً مقدار -21 که در ستون آخر PDF آمده، در رجیستر فایل ثبت نمی‌شد). همچنین احتمال وجود اشکال در بخش Val2Gen و یا حتی در مرحله Instruction Decode نیز وجود دارد.

جلسه پنجم - 7 اردیبهشت 1404

در این جلسه، کد ساخته شده را روی سخت افزار بردیم و خروجی گرفتیم، همچنین مشکلاتی که از جلسات قبل داشتیم را اصلاح کردیم

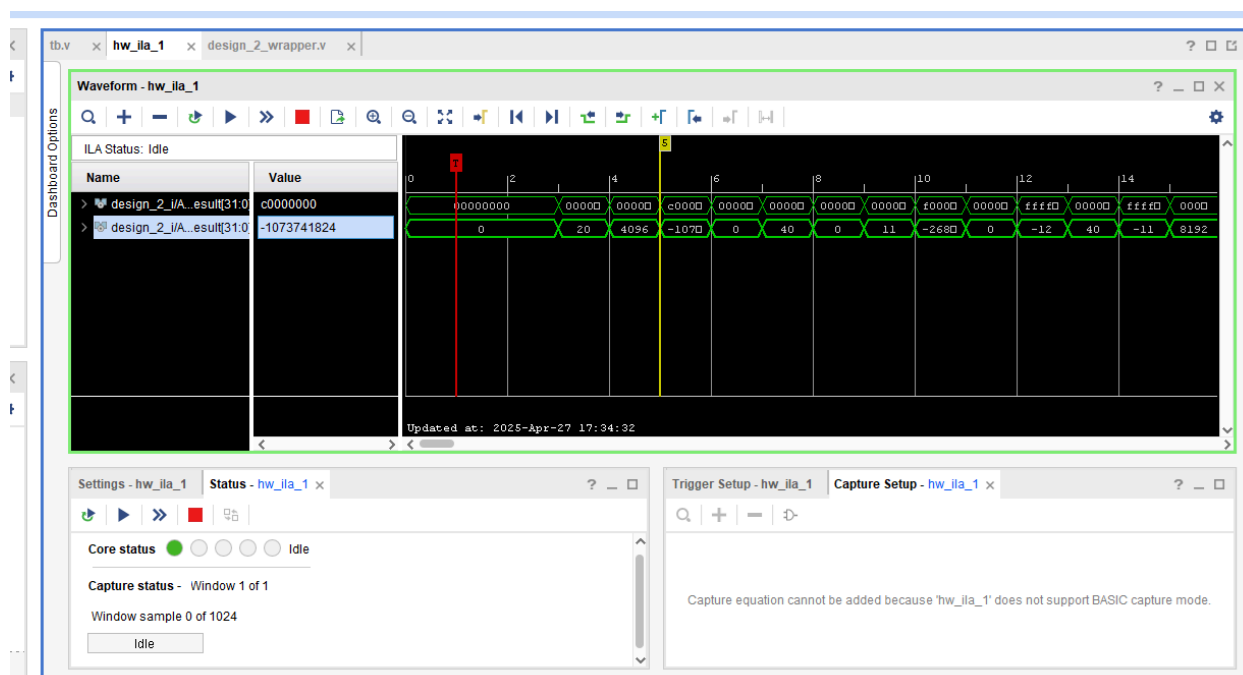
1. مشکل صفر بودن خروجی ALU از جلسه قبل

برای بند 7 در جلسه پیش، کد مشکلی نداشت، دستور هازارد دارد، از آنجا که هنوز مازول Hazard Detector نداریم، باعث می شود نتیجه خروجی صفر ببینیم

2. نصب روی نرم افزار و خروجی:

برای انتقال کد به نرم افزار، طبق درخواست TA ها:

- خروجی ALU را به ILA probe وصل کردین به همراه trigger تا در خروجی ببینیم
- تریگر (همان ریست) را از debouncer گذراندیم (مثل قبل)
- بیت استریم ساخته و program کردیم



شکل. نتیجه خروجی در سخت افزار

تحلیل خروجی:

- نتیجه روی سخت افزار تا 3 خروجی به درستی آمده اند، چون هنوز Hazard Unit نداریم، دستورات بعدی مهم نیستند و جزو جلسات آینده خواهند بود (دستور 4 باعث هازارد می شود)

جلسه ششم - 14 اردیبهشت 1404

در این جلسه Hazard Unit وصل شد و تعداد زیادی مشکل به رو آمدند، به دلیل کم بودن زمان، تعدادی از مشکلات خارج از جلسه کلاسی حل شدند (با هماهنگی TA). انواع هازارد ها داریم: RAW, WAW و WAR، طبق توضیحات صورت پروژه، فقط RAW در پیاده سازی ما رخ می دهد و باید در ماژول Hazard Unit از آن جلوگیری شود.

به طور کلی این ماژول، سیگنال Freeze را در حالات مختلف emit می کند که باعث می شود دستوری از instruction memory خوانده نشود و دستوراتی که ممکن است مخاطره ایجاد کنند، تمام شوند، trade off زمان می دهیم تا سخت افزار به درستی کار کند

هازارد در واقع به خاطر وابستگی داده ای رخ می دهد، برای مثال خروجی یک دستور، ورودی دستور دقیقاً بعدی نیز می باشد، اما ممکن است دستور بعدی، قبل اتمام دستور کنونی (نوشته شدن در رجیستر یا Memory) وارد پایپلاین شود و داده اشتباهی بخواند، به خاطر همین موضوع کد سیگنال Freeze صادر می شود.

1. مشکل سلکتور:

سلکتور رجیستر cmux برعکس داشت کار می کرد که درست کردیم (برعکس)

2. مشکل Carry در ALU:

alu مقدار Carry را درست حساب نمیکرد که درست کردیم، موقع جمع، باید carry in را به 32 بیت اکستند می کردیم در ALU تا این مشکل حل شود

3. مقدار s باید [20] instruction می بود که درست محاسبه نمی شد

4. مشکل لوپ combinational:

ماژول OR برای two src که به هازارد unit وصل می شد، mem write enable را از ماکس بعد کنترلر می گرفت و باعث combinational loop می شد، برای حل این مشکل mem write enable را از قبل ماکس گرفتیم، Imm نیز چنین مشکلی داشت که رفع شد

با افزوده شدن Hazard unit الان نتایج فراتر از نتیجه سوم هم در مموری نوشته می شوند، در بررسی اولیه، تا نتیجه 11 ام درست بود (12 سطر مموری از صفحه 17 گزارش)

در بررسی های بیشتر، متوجه شدیم تا مورد 35 درست اجرا می شود، بعدش در یک دور خاصی بخشی از خروجی های Pipeline مقدار X می گیرند. اددر در مرحله exe (قرمز) ورودی a باید 24 بیتی و signed می بود، چون اینطور نبود آدرس Branch مقدار X می گرفت و خراب می شد.

بعد حل مشکل بالا، دستورات تا جای مورد نیاز به درستی اجرا می شدند، همچنین صورت پروژه یک اشکالی دارد، دستور 46 نوشته رجیستر 4 مقدارش میشود 10 در حالی که طبق دستور مشخص است که باید رجیستر 6 مقدارش 10 شود (خروجی پروژه ما به درستی در آدرس ششم قرار می دهد)

R6[R0],#20

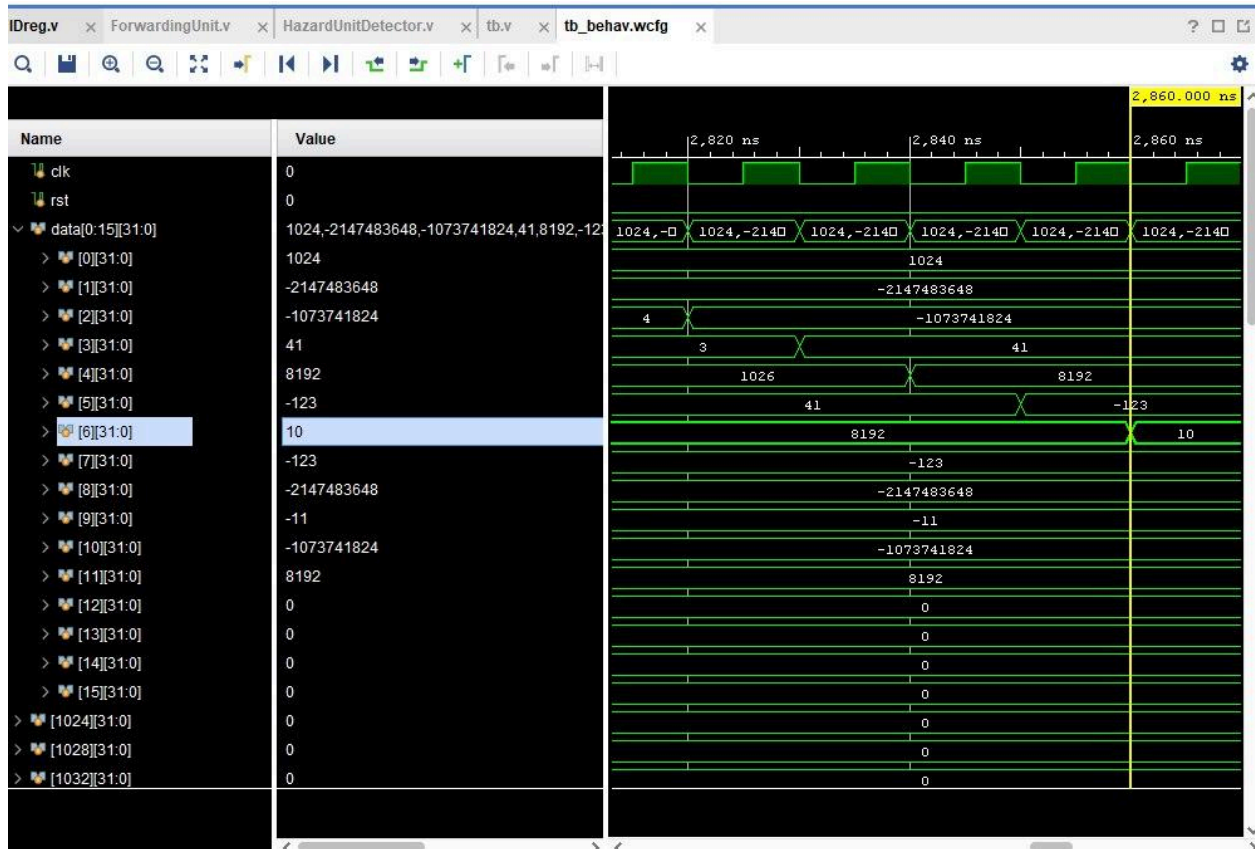
R4 = 10

data[0:15][31:0]	1024,-2147483648,-1073741824,41,8192
> [0][31:0]	1024
> [1][31:0]	-2147483648
> [2][31:0]	-1073741824
> [3][31:0]	41
> [4][31:0]	8192
> [5][31:0]	-123
> [6][31:0]	10
> [7][31:0]	-123
> [8][31:0]	-2147483648
> [9][31:0]	-11
> [10][31:0]	-1073741824
> [11][31:0]	8192
> [12][31:0]	0
> [13][31:0]	0
> [14][31:0]	0
> [15][31:0]	0
> [1024][31:0]	0

شکل. مموری بعد اتمام دستورات

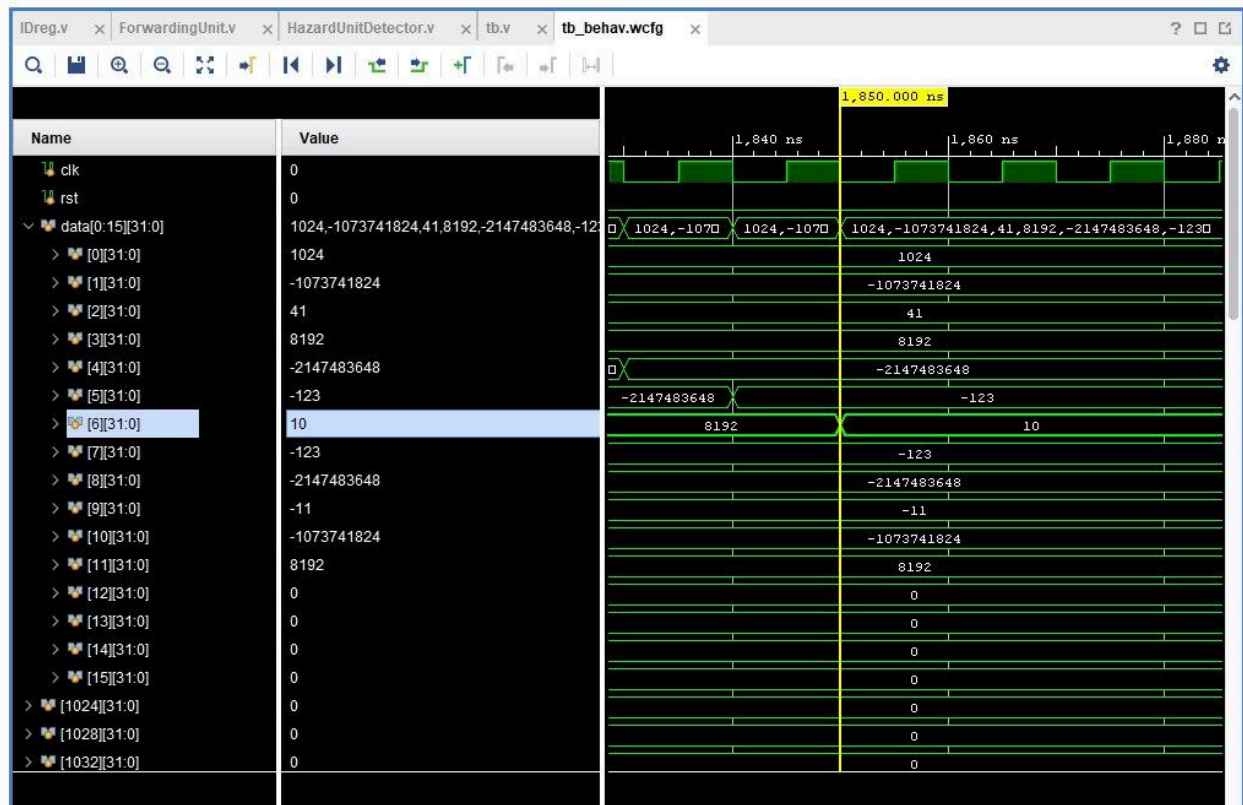
جلسه هفتم - 28 اردیبهشت 1404

در این مرحله، ARM را روی سخت افزار بردیم، ورودی write رجیستر فایل به ILA وصل شد و مقادیر توسط TA تایید شدند، واحد فورواردر نیز در این جلسه اضافه شد، هدف این مازول این است که با منتقل کردن سریع تر دیتا تا جای ممکن Hazard ها را کمتر کند (وابستگی داده باعث freeze نشود) و در نتیجه پردازنده سریع تر نیز شود.



شکل.نتیجه شبیه سازی بدون فورواردر

با فورواردر نزدیک به 1000 نانوثانیه سریع تر است (در جلسه بعد این عدد اصلاح می شود، مشکل ریزی در پیاده سازی کنونی وجود دارد):



اگر به اندیس 4 مموری (دیتا نگاه کنیم) با فعال شدن forwarding unit عوض شده و اشتباه است، این مشکل در جلسه بعد حل شده

گزارش سخت افزار استفاده شده بعد افزودن فوروارد:

Tcl Console Messages Log Reports Design Runs x																	?		🔍 🗑	
Name	Constraints	Status	WNS	TNS	WHS	THS	TPWS	Total Power	Failed Routes	LUT	FF	BRAMs	URAM	DSP	Start	Elapsed	Run Strategy			
impl_1	constrs_1	Not started															Vivado Implementation I			
Out-of-Context Module Runs																				
design_1		Submodule Runs Complete													3/9/25, 6:00 PM	00:41:24				
design_2		Submodule Runs Complete													3/9/25, 6:58 PM	1,678:54:44				
design_2_Mux_0_synth_1	design_2_	synth_design Complete								16	0	0.00	0	0	3/9/25, 6:58 PM	00:00:45	Vivado Synthesis Defau			
design_2_PC_0_synth_1	design_2_	synth_design Complete								1	32	0.00	0	0	3/9/25, 6:58 PM	00:00:29	Vivado Synthesis Defau			
design_2_debouncer_0_0_synth_1	design_2_	synth_design Complete								7	17	0.00	0	0	3/9/25, 6:58 PM	00:00:29	Vivado Synthesis Defau			
design_2_FReg_0_synth_1	design_2_	synth_design Complete								33	64	0.00	0	0	4/6/25, 4:13 PM	00:00:40	Vivado Synthesis Defau			
design_2_util_vector_logic_0_0_synth_1	design_2_	synth_design Complete								1	0	0.00	0	0	4/13/25, 5:25 PM	00:00:51	Vivado Synthesis Defau			
design_2_util_vector_logic_1_0_synth_1	design_2_	synth_design Complete								1	0	0.00	0	0	4/13/25, 5:25 PM	00:00:51	Vivado Synthesis Defau			
design_2_ReglMux_0_0_synth_1	design_2_	synth_design Complete								2	0	0.00	0	0	4/13/25, 5:25 PM	00:00:50	Vivado Synthesis Defau			
design_2_dst_mem_gen_0_0_synth_1	design_2_	synth_design Complete								59	0	0.00	0	0	5/4/25, 4:41 PM	00:03:38	Vivado Synthesis Defau			
design_2_dst_mem_gen_1_0_synth_1	design_2_	synth_design Complete								4416	32	0.00	0	0	4/20/25, 5:25 PM	00:01:30	Vivado Synthesis Defau			
design_2_util_vector_logic_0_1_synth_1	design_2_	synth_design Complete								8	0	0.00	0	0	4/20/25, 5:25 PM	00:01:11	Vivado Synthesis Defau			
design_2_ExeReg_0_0_synth_1	design_2_	synth_design Complete								0	71	0.00	0	0	4/20/25, 5:25 PM	00:01:12	Vivado Synthesis Defau			
design_2_Mux_1_synth_1	design_2_	synth_design Complete								16	0	0.00	0	0	4/20/25, 5:58 PM	00:00:30	Vivado Synthesis Defau			
design_2_MEM_Stage_Reg_0_0_synth_1	design_2_	synth_design Complete								0	70	0.00	0	0	4/20/25, 5:58 PM	00:00:30	Vivado Synthesis Defau			
design_2_Val2Gen_0_0_synth_1	design_2_	synth_design Complete								345	0	0.00	0	0	4/27/25, 5:09 PM	00:00:30	Vivado Synthesis Defau			
design_2_ila_0_0_synth_1	design_2_	synth_design Complete								911	1744	6.50	0	0	5/18/25, 5:50 PM	00:02:17	Vivado Synthesis Defau			
design_2_Adder_0_synth_1	design_2_	synth_design Complete								32	0	0.00	0	0	5/11/25, 11:30 ...	00:01:15	Vivado Synthesis Defau			
design_2_CUMUX_0_0_synth_1	design_2_	synth_design Complete								10	0	0.00	0	0	5/11/25, 11:30 ...	00:01:14	Vivado Synthesis Defau			
design_2_ConditionCheck_0_0_synth_1	design_2_	synth_design Complete								4	0	0.00	0	0	5/11/25, 11:30 ...	00:01:16	Vivado Synthesis Defau			
design_2_ControlUnit_0_0_synth_1	design_2_	synth_design Complete								10	0	0.00	0	0	5/11/25, 11:30 ...	00:01:14	Vivado Synthesis Defau			
design_2_StatusReg_0_0_synth_1	design_2_	synth_design Complete								0	4	0.00	0	0	5/11/25, 11:30 ...	00:01:15	Vivado Synthesis Defau			
design_2_ALU_0_0_synth_1	design_2_	synth_design Complete								217	0	0.00	0	0	5/11/25, 11:30 ...	00:01:16	Vivado Synthesis Defau			
design_2_Red4Adder_0_0_synth_1	design_2_	synth_design Complete								32	0	0.00	0	0	5/11/25, 11:30 ...	00:01:14	Vivado Synthesis Defau			
design_2_DReg_0_0_synth_1	design_2_	synth_design Complete								79	158	0.00	0	0	5/18/25, 5:03 PM	00:00:54	Vivado Synthesis Defau			
design_2_HazardUnitDefector_0_0_synth_1	design_2_	synth_design Complete								7	0	0.00	0	0	5/18/25, 5:03 PM	00:00:54	Vivado Synthesis Defau			
design_2_Exe_Mux_0_0_synth_1	design_2_	synth_design Complete								32	0	0.00	0	0	5/18/25, 5:03 PM	00:00:54	Vivado Synthesis Defau			
design_2_Exe_Mux_0_1_synth_1	design_2_	synth_design Complete								32	0	0.00	0	0	5/18/25, 5:03 PM	00:00:54	Vivado Synthesis Defau			
design_2_ForwardingUnit_0_0_synth_1	design_2_	synth_design Complete								9	0	0.00	0	0	5/18/25, 5:19 PM	00:00:32	Vivado Synthesis Defau			
design_2_RegisterFile_0_0_synth_1	design_2_	synth_design Complete								272	512	0.00	0	0	5/18/25, 5:50 PM	00:00:35	Vivado Synthesis Defau			
design_2_util_vector_logic_0_2		Using cached IP results																		
design_2_util_vector_logic_0_3		Using cached IP results																		

از پایین ویوادر بخش design runs می تواند گزارش سخت افزار گرفت، مقادیر بالا ممکن است کمی دقیق نباشند، چون تعدادی NOT بعدا به خاطر مشکلی در Hazard Unit جابه جا شدند (جلسه بعدی) و ماژول VIO نیز اضافه شد

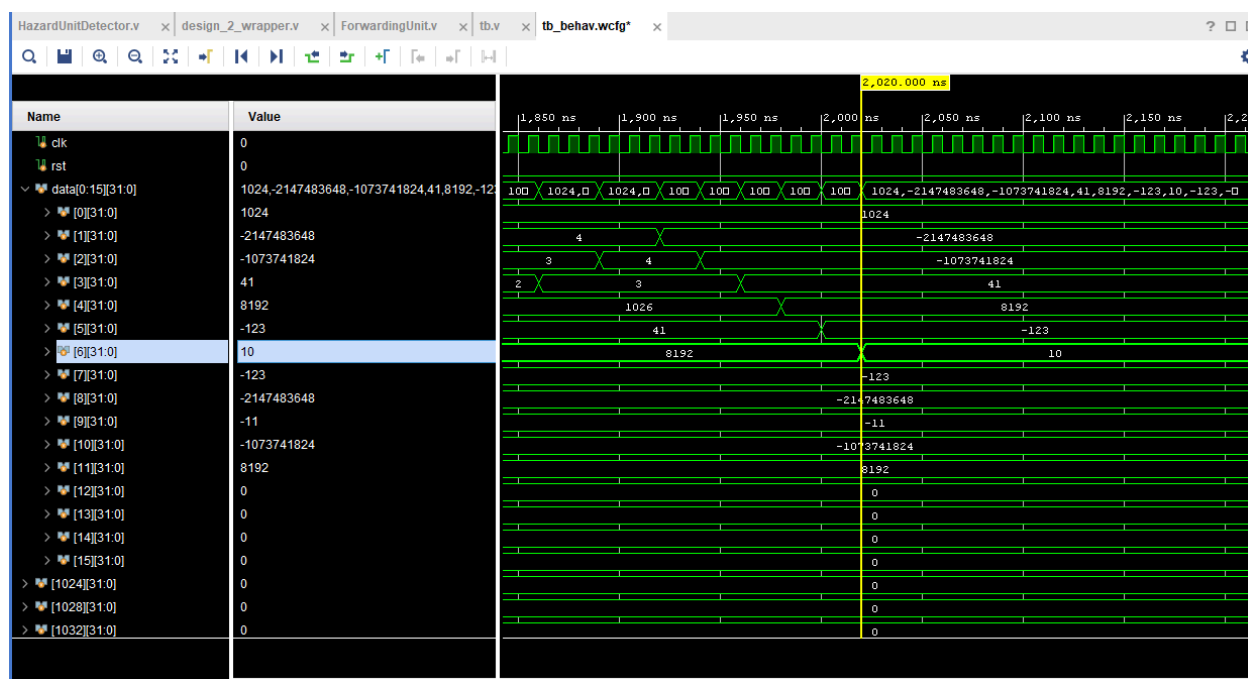
جلسه هشتم - 4 خرداد 1404

در این جلسه، به رفع مشکلات نهایی پرداختیم، یک مشکلی که در wave form های با forwarding unit بود که بخش کوچکی درست نبودند

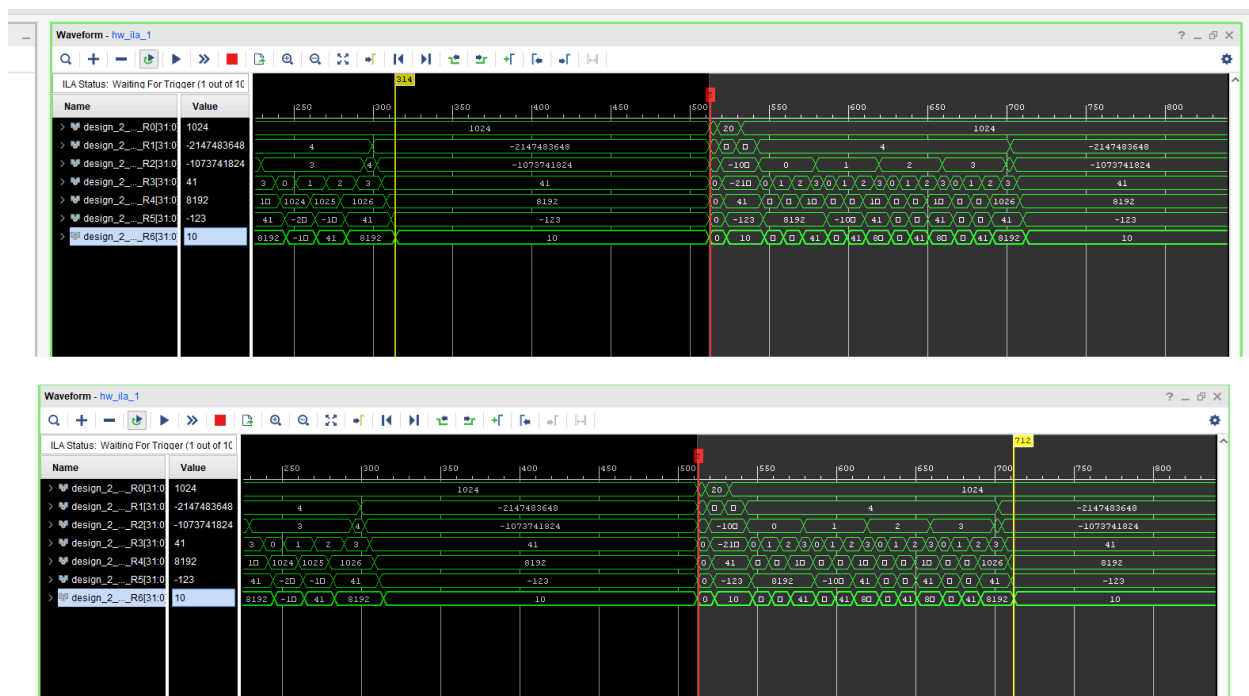
مشکل اصلی به Hazard Detector بر می گشت، از `is_two_src` استفاده کرده بودیم که نات پشت آن اشتباه بود، با رفع این مشکل خطای نتایج forwarding unit برطرف شد

نات داشتن یا نداشتن این مورد، فرقی در نتایج بدون forwarding unit نداشت (یعنی بدون فوروارد درست بودند) که در نوع خود عجیب بود، با همفکری TA ها به این نتیجه رسیدیم که دستوراتی که وابسته به دستورات قبل بودند و two src هم هستند، شرط اولیه را در هازارد یونیت برقرار می کنند و بخشی از Hazard unit که اشتباه بود هیچ وقت اجرا نمی شد (در حالت بدون ماژول فوروارد)، در نتیجه خطایی برای آن قبلا مشاهده نکرده بودیم

نسخه درست شده بعد حذف نات ها، که 850 نانوثانیه سریع تر است:



سپس هر دو حالت را روی سخت افزار بردیم، امواج نهایی، هر دو یکی هستند، بالایی اتمام بدون فوروارد را نشان می دهد، پایینی با فوروارد:



دومی از 512 شروع شده با توجه به window width تنظیم شده

از ابزاری به نام vio یا همان virtual input output برای کنترل forward enabled در سخت افزار کردیم (از طریق نرم افزار برد را کنترل می کنیم با vio)